# Chapter 16
# JavaFX UI Controls and Multimedia


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Motivations

A graphical user interface (GUI) makes a system user-friendly and easy to use. Creating a GUI requires creativity and knowledge of how GUI components work. Since the GUI components in Java are very flexible and versatile, you can create a wide assortment of useful user interfaces.

Previous chapters briefly introduced several GUI components. This chapter introduces the frequently used GUI components in detail.

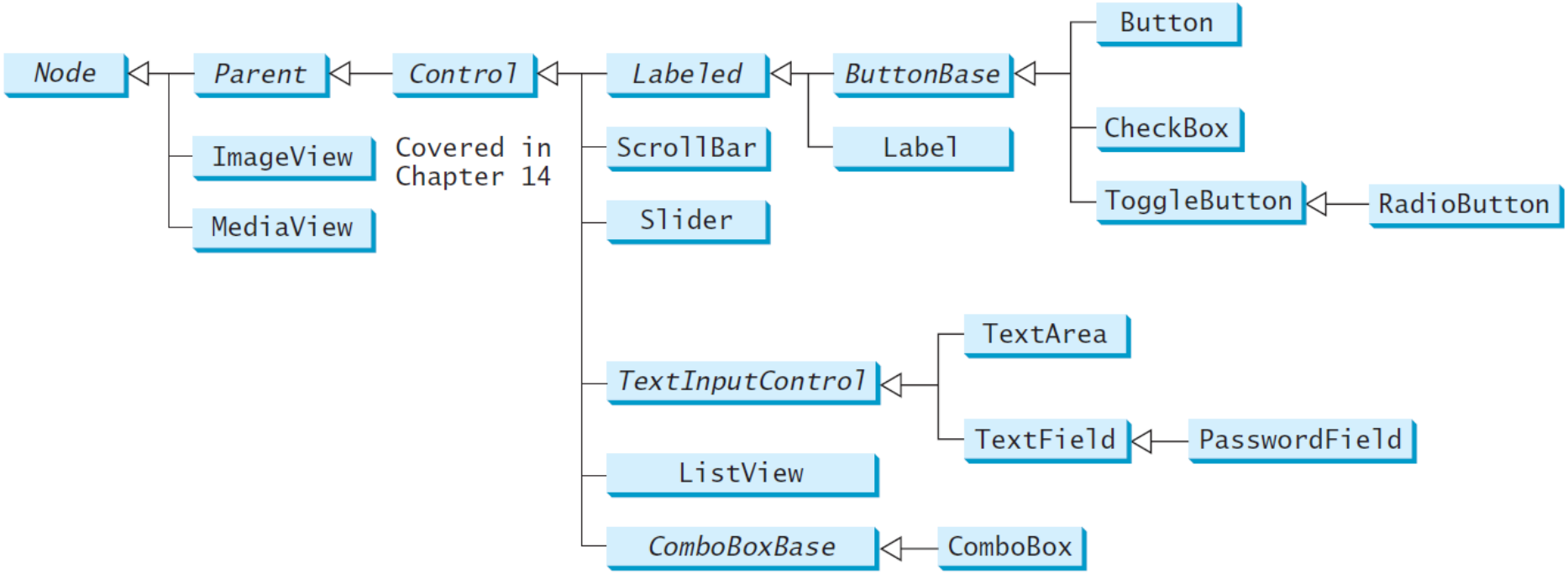

Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Objectives

- To create graphical user interfaces with various user-interface controls (§§16.2–16.11).
- To create a label with text and graphic using the **Label** class and explore properties in the abstract **Labeled** class (§16.2).
- To create a button with text and graphic using the **Button** class and set a handler using the **setOnAction** method in the abstract **ButtonBase** class (§16.3).
- To create a check box using the **CheckBox** class (§16.4).
- To create a radio button using the **RadioButton** class and group radio buttons using a **ToggleGroup** (§16.5).
- To enter data using the **TextField** class and password using the **PasswordField** class (§16.6).
- To enter data in multiple lines using the **TextArea** class (§16.7).
- To select a single item using **ComboBox** (§16.8).
- To select a single or multiple items using **ListView** (§16.9).
- To select a range of values using **ScrollBar** (§16.10).
- To select a range of values using **Slider** and explore differences between **ScrollBar** and **Slider** (§16.11).
- To develop a tic-tac-toe game (§16.12).
- To view and play video and audio using the **Media**, **MediaPlayer**, and **MediaView** (§16.13).
- To develop a case study for showing the national flag and play anthem (§16.14).


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Frequently Used UI Controls




Throughout this book, the prefixes **lbl**, **bt**, **chk**, **rb**, **tf**, **pf**, **ta**, **cbo**, **lv**, **scb**, **sld**, and **mp** are used to name reference variables for **Label**, **Button**, **CheckBox**, **RadioButton**, **TextField**, **PasswordField**, **TextArea**, **ComboBox**, **ListView**, **ScrollBar**, **Slider**, and **MediaPlayer**.


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Labeled

A *label* is a display area for a short text, a node, or both. It is often used to label other controls (usually text fields). Labels and buttons share many common properties. These common properties are defined in the **Labeled** class.

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

***javafx.scene.control.Labeled***

```
-alignment: ObjectProperty<Pos>
-contentDisplay:
   ObjectProperty<ContentDisplay>
-graphic: ObjectProperty<Node>
-graphicTextGap: DoubleProperty
-textFill: ObjectProperty<Paint>
-text: StringProperty
-underline: BooleanProperty
-wrapText: BooleanProperty
```

Specifies the alignment of the text and node in the labeled.

Specifies the position of the node relative to the text using the constants `TOP`, `BOTTOM`, `LEFT`, and `RIGHT` defined in `ContentDisplay`.

A graphic for the labeled.

The gap between the graphic and the text.

The paint used to fill the text.

A text for the labeled.

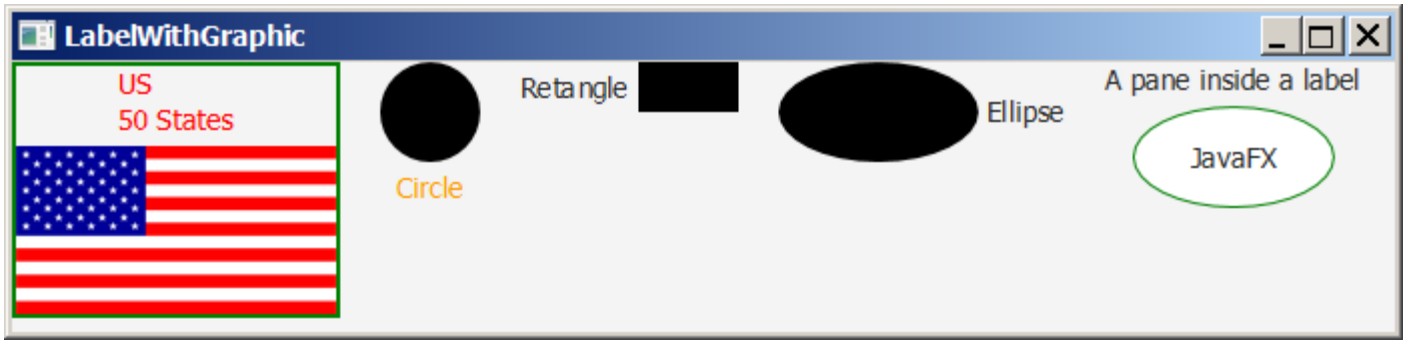
Whether text should be underlined.

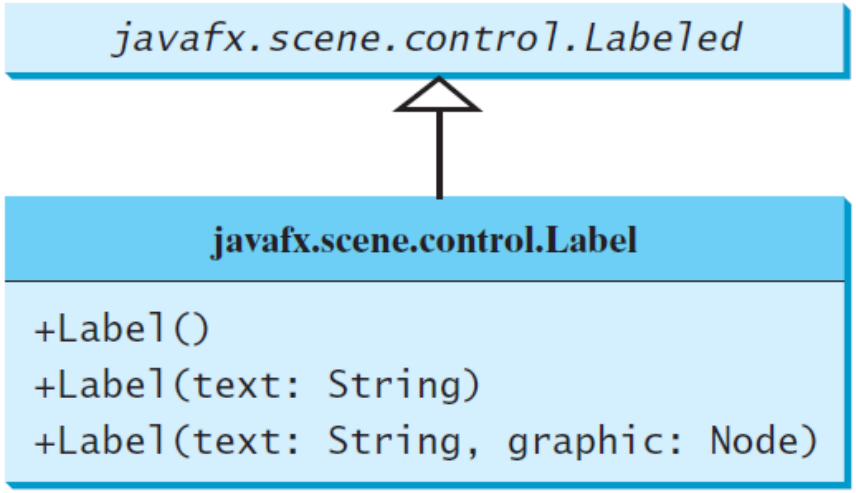
Whether text should be wrapped if the text exceeds the width.


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Label

The Label class defines labels.



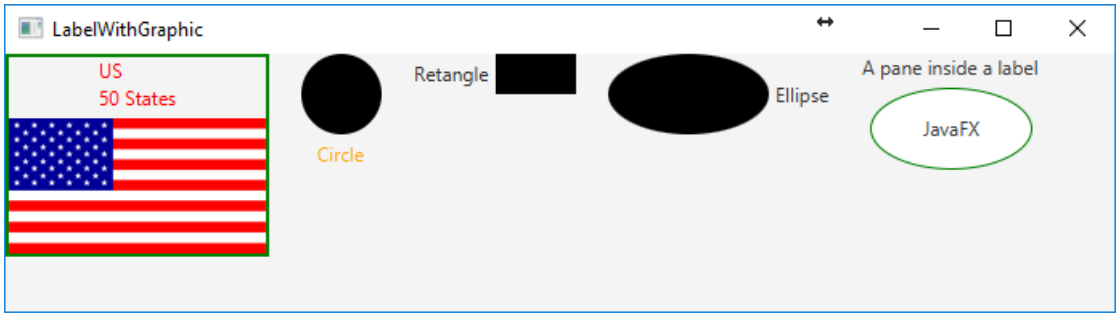




| | |
|---|---|
| Creates an empty label. | |
| Creates a label with the specified text. | |
| Creates a label with the specified text and graphic. | |

LabelWithGraphic


Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.ContentDisplay;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.HBox;
import javafx.scene.layout.StackPane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.shape.Rectangle;
import javafx.scene.shape.Ellipse;
public class LabelWithGraphic extends Application {
  @Override
  public void start(Stage primaryStage) {
    ImageView us = new ImageView(new Image("image/us.gif"));
    Label lb1 = new Label("US\n50 States", us);
    lb1.setStyle("-fx-border-color: green; -fx-border-width: 2");
    lb1.setContentDisplay(ContentDisplay.BOTTOM);
    lb1.setTextFill(Color.RED);
    Label lb2 = new Label("Circle", new Circle(50, 50, 25));
    lb2.setContentDisplay(ContentDisplay.TOP);
    lb2.setTextFill(Color.ORANGE);
    Label lb3 = new Label("Retangle", new Rectangle(10, 10, 50, 25));
    lb3.setContentDisplay(ContentDisplay.RIGHT);
    Label lb4 = new Label("Ellipse", new Ellipse(50, 50, 50, 25));
    lb4.setContentDisplay(ContentDisplay.LEFT);
```





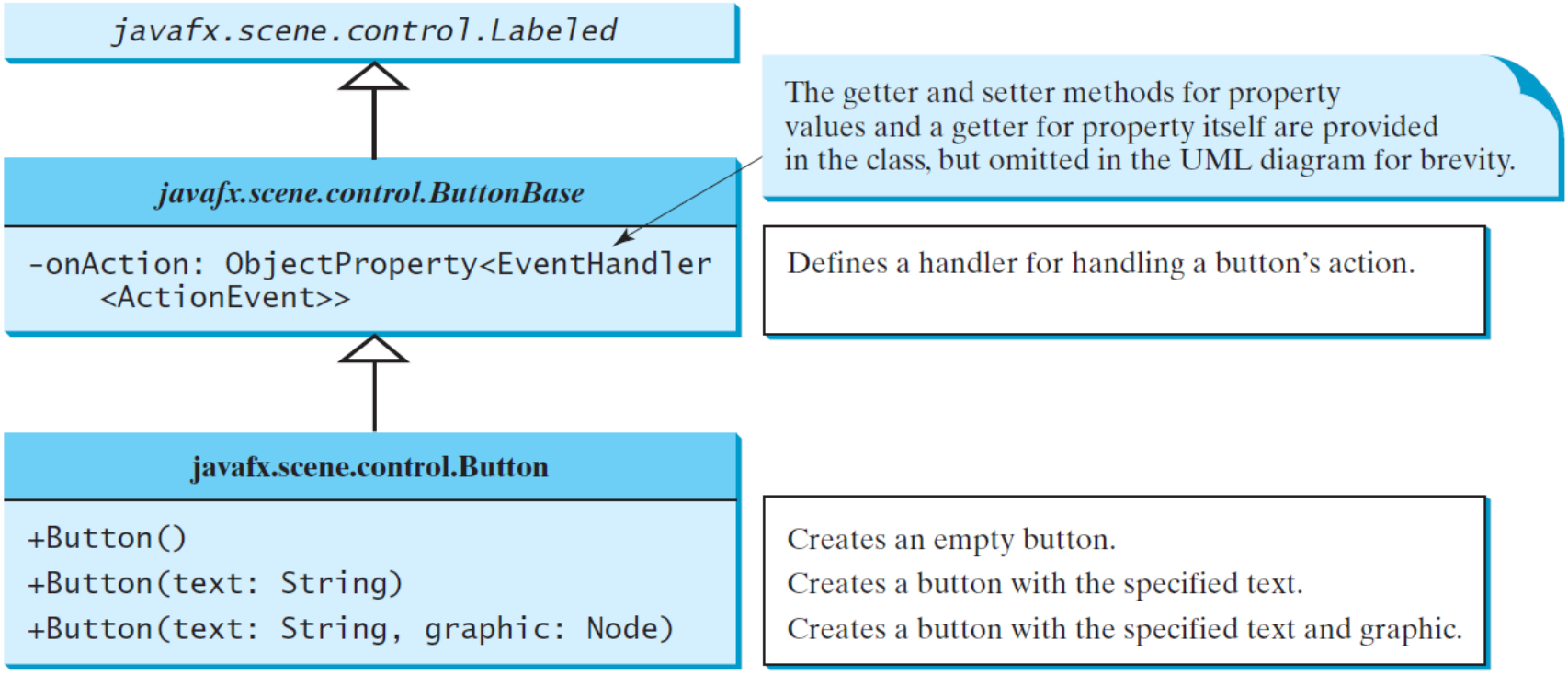
Copyright © 2022 Pearson Education Ltd. All rights reserved.

```
    Ellipse ellipse = new Ellipse(50, 50, 50, 25);
    ellipse.setStroke(Color.GREEN);
    ellipse.setFill(Color.WHITE);
    StackPane stackPane = new StackPane();
    stackPane.getChildren().addAll(ellipse, new  Label("JavaFX"));
    Label lb5 = new Label("A pane inside a label", stackPane);
    lb5.setContentDisplay(ContentDisplay.BOTTOM);
    HBox pane = new HBox(20);
    pane.getChildren().addAll(lb1, lb2, lb3, lb4, lb5);
    Scene scene = new Scene(pane, 700, 150);
    primaryStage.setTitle("LabelWithGraphic");
    primaryStage.setScene(scene);
    primaryStage.show();
  }
  public static void main(String[] args) {
     launch(args);
  }
}
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# ButtonBase and Button

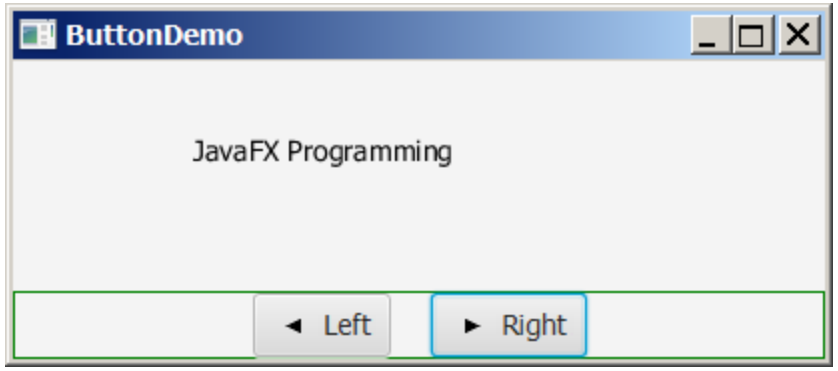
A *button* is a control that triggers an action event when clicked. JavaFX provides regular buttons, toggle buttons, check box buttons, and radio buttons. The common features of these buttons are defined in **ButtonBase** and **Labeled** classes.





Copyright © 2022 Pearson Education Ltd. All rights reserved.

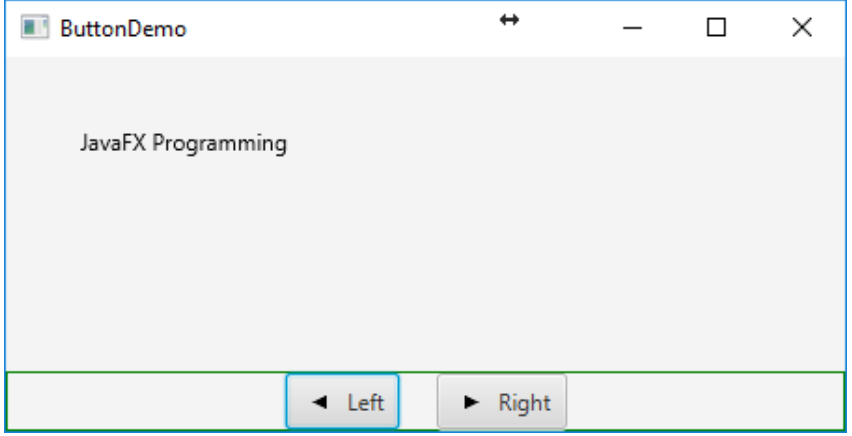
# Button Example




ButtonDemo


Copyright © 2022 Pearson Education Ltd. All rights reserved.

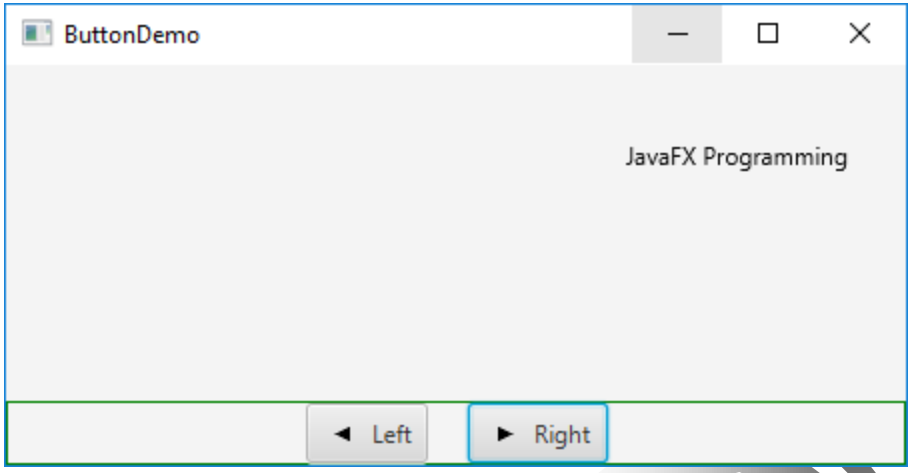
```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.image.ImageView;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Pane;
import javafx.scene.text.Text;
```




```java
public class ButtonDemo extends Application {
  protected Text text = new Text(50, 50, "JavaFX Programming");
  protected BorderPane getPane() {
    HBox paneForButtons = new HBox(20);
    Button btLeft = new Button("Left", new ImageView("image/left.gif"));
    Button btRight = new Button("Right", new ImageView("image/right.gif"));
    paneForButtons.getChildren().addAll(btLeft, btRight);
    paneForButtons.setAlignment(Pos.CENTER);
    paneForButtons.setStyle("-fx-border-color: green");
    BorderPane pane = new BorderPane();
    pane.setBottom(paneForButtons);
    Pane paneForText = new Pane();
    paneForText.getChildren().add(text);
    pane.setCenter(paneForText);

    btLeft.setOnAction(e -> text.setX(text.getX() - 10));
    btRight.setOnAction(e -> text.setX(text.getX() + 10));

    return pane;
  }
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

```
  @Override
  public void start(Stage primaryStage) {
    Scene scene = new Scene(getPane(), 450, 200);
    primaryStage.setTitle("ButtonDemo");
    primaryStage.setScene(scene);
    primaryStage.show();
  }
  public static void main(String[] args) {
    launch(args);
  }

}
```

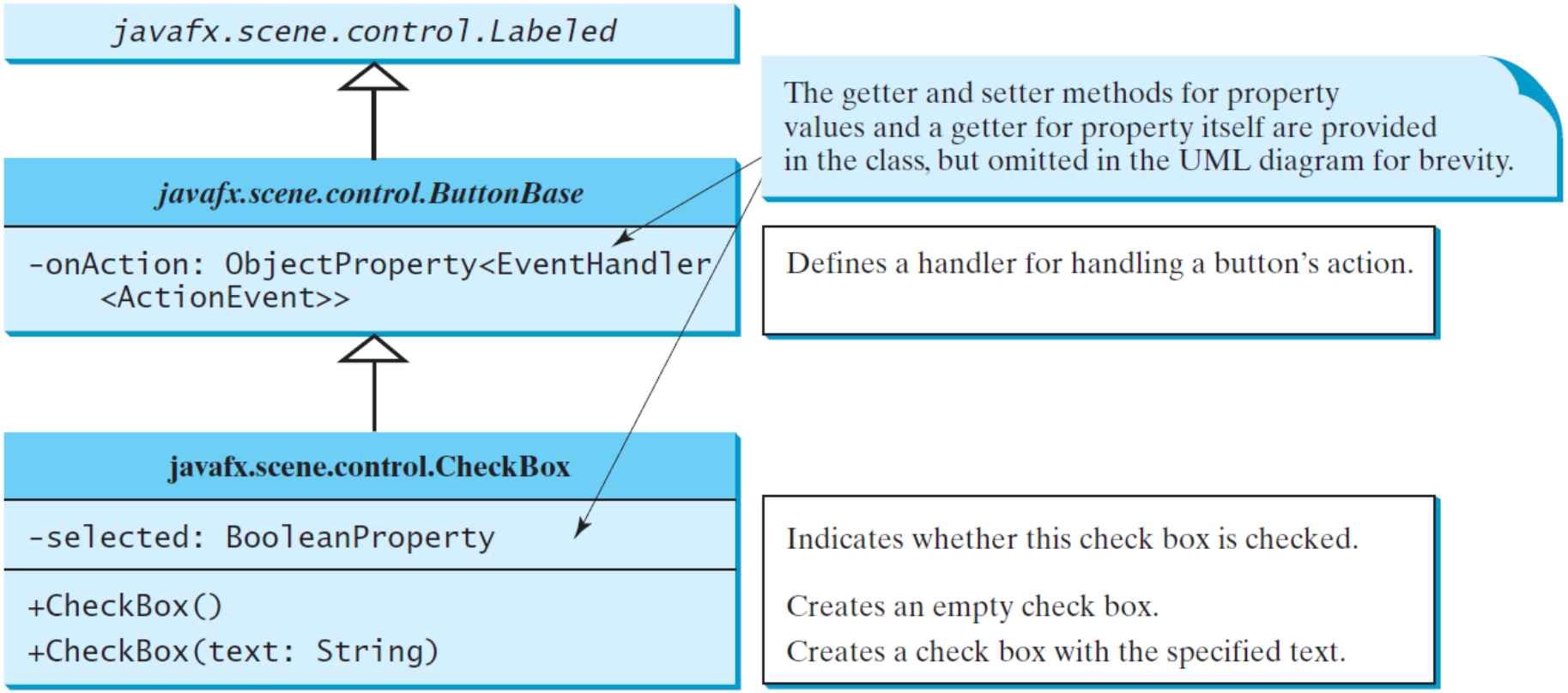




Copyright © 2022 Pearson Education Ltd. All rights reserved.

# CheckBox

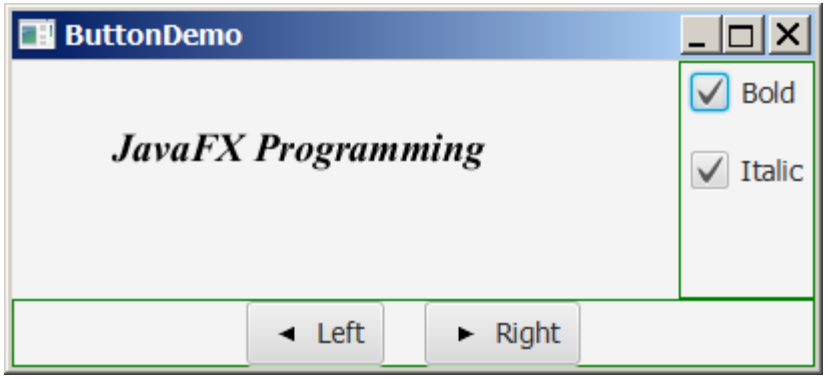
A **CheckBox** is used for the user to make a selection. Like **Button**, **CheckBox** inherits all the properties such as **onAction**, **text**, **graphic**, **alignment**, **graphicTextGap**, **textFill**, **contentDisplay** from **ButtonBase** and **Labeled**.





Copyright © 2022 Pearson Education Ltd. All rights reserved.

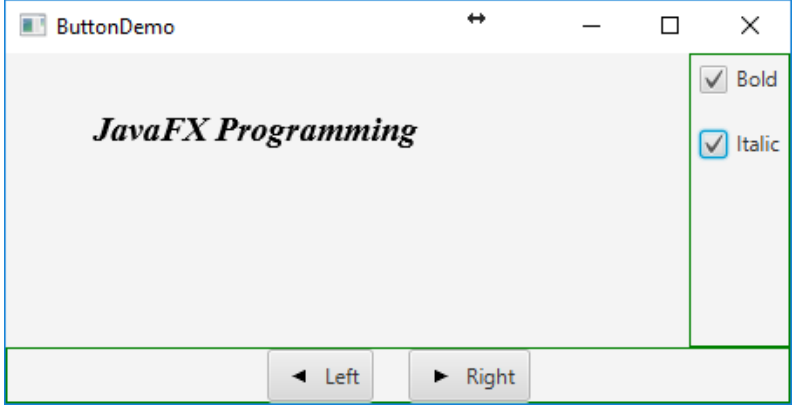
# CheckBox Example




CheckBoxDemo


Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.geometry.Insets;
import javafx.scene.control.CheckBox;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.VBox;
import javafx.scene.text.Font;
import javafx.scene.text.FontPosture;
import javafx.scene.text.FontWeight;

public class CheckBoxDemo extends ButtonDemo {
  @Override // Override the getPane() method in the super class
  protected BorderPane getPane() {
    BorderPane pane = super.getPane();

    Font fontBoldItalic = Font.font("Times New Roman",
    FontWeight.BOLD, FontPosture.ITALIC, 20);
    Font fontBold = Font.font("Times New Roman",
    Font Weight.BOLD, FontPosture.REGULAR, 20);
    Font fontItalic = Font.font("Times New Roman",
    FontWeight.NORMAL, FontPosture.ITALIC, 20);
    Font fontNormal = Font.font("Times New Roman",
    FontWeight.NORMAL, FontPosture.REGULAR, 20);

    text.setFont(fontNormal);

    VBox paneForCheckBoxes = new VBox(20);
    paneForCheckBoxes.setPadding(new Insets(5, 5, 5, 5));
    paneForCheckBoxes.setStyle("-fx-border-color: green");
```





Copyright © 2022 Pearson Education Ltd. All rights reserved.

```
    CheckBox chkBold = new CheckBox("Bold");
    CheckBox chkItalic = new CheckBox("Italic");
    paneForCheckBoxes.getChildren().addAll(chkBold, chkItalic);
    pane.setRight(paneForCheckBoxes);

    EventHandler<ActionEvent> handler = e -> {
       if (chkBold.isSelected() && chkItalic.isSelected()) {
          text.setFont(fontBoldItalic); // Both check boxes checked
       } else if (chkBold.isSelected()) {
          text.setFont(fontBold); // The Bold check box checked
       } else if (chkItalic.isSelected()) {
          text.setFont(fontItalic); // The Italic check box checked
       } else {
           text.setFont(fontNormal); // Both check boxes unchecked
       }
     };

     chkBold.setOnAction(handler);
     chkItalic.setOnAction(handler);

     return pane; // Return a new pane
  }

  // the start method is inherited from the superclass ButtonDemo
  public static void main(String[] args) {
     launch(args);
  }

}
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# RadioButton

Radio buttons, also known as *option buttons*, enable you to choose a single item from a group of choices. In appearance radio buttons resemble check boxes, but check boxes display a square that is either checked or blank, whereas radio buttons display a circle that is either filled (if selected) or blank (if not selected).

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

**javafx.scene.control.ToggleButton**

```
-selected: BooleanProperty
-toggleGroup:
   ObjectProperty<ToggleGroup>
```

Indicates whether the button is selected.

Specifies the button group to which the button belongs.

```
+ToggleButton()
+ToggleButton(text: String)
+ToggleButton(text: String, graphic: Node)
```

Creates an empty toggle button.

Creates a toggle button with the specified text.

Creates a toggle button with the specified text and graphic.

**javafx.scene.control.RadioButton**

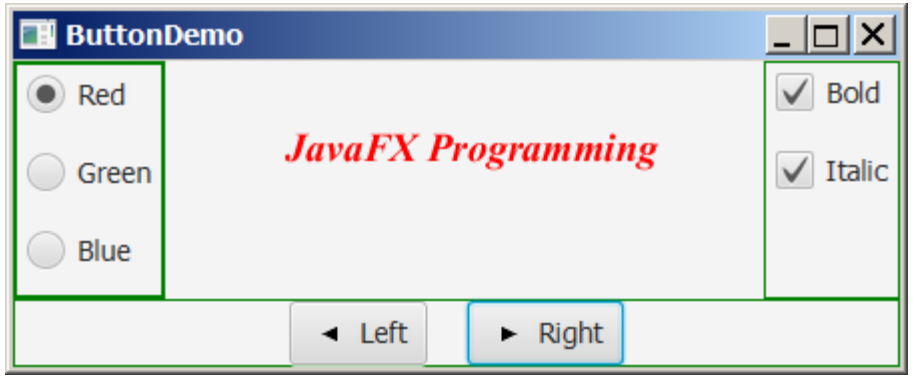
```
+RadioButton()
+RadioButton(text: String)
```

Creates an empty radio button.

Creates a radio button with the specified text.

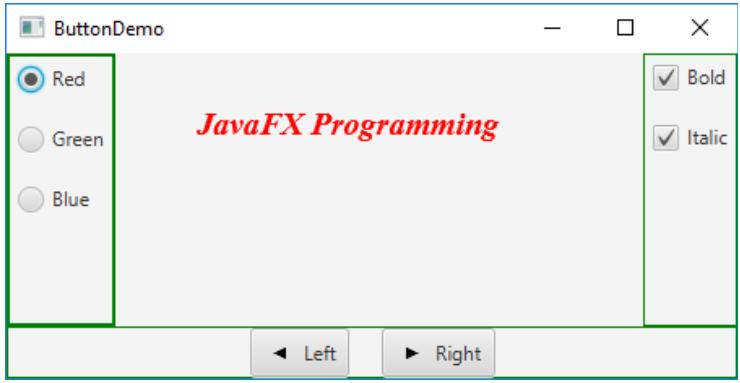

Copyright © 2022 Pearson Education Ltd. All rights reserved.

# RadioButton Example




RadioButtonDemo


Copyright © 2022 Pearson Education Ltd. All rights reserved.




```
import static javafx.application.Application.launch;
import javafx.geometry.Insets;
import javafx.scene.control.RadioButton;
import javafx.scene.control.ToggleGroup;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;

public class RadioButtonDemo extends CheckBoxDemo {
  @Override // Override the getPane() method in the super class
  protected BorderPane getPane() {
    BorderPane pane = super.getPane();
    VBox paneForRadioButtons = new VBox(20);
    paneForRadioButtons.setPadding(new Insets(5, 5, 5, 5));
    paneForRadioButtons.setStyle("-fx-border-color: green");
    RadioButton rbRed = new RadioButton("Red");
    RadioButton rbGreen = new RadioButton("Green");
    RadioButton rbBlue = new RadioButton("Blue");
    paneForRadioButtons.getChildren().addAll(rbRed, rbGreen, rbBlue);
    pane.setLeft(paneForRadioButtons);
    ToggleGroup group = new ToggleGroup();
    rbRed.setToggleGroup(group);
    rbGreen.setToggleGroup(group);
    rbBlue.setToggleGroup(group);

    rbRed.setOnAction(e -> {
      if (rbRed.isSelected()) {
        text.setFill(Color.RED);
      }

    });
```

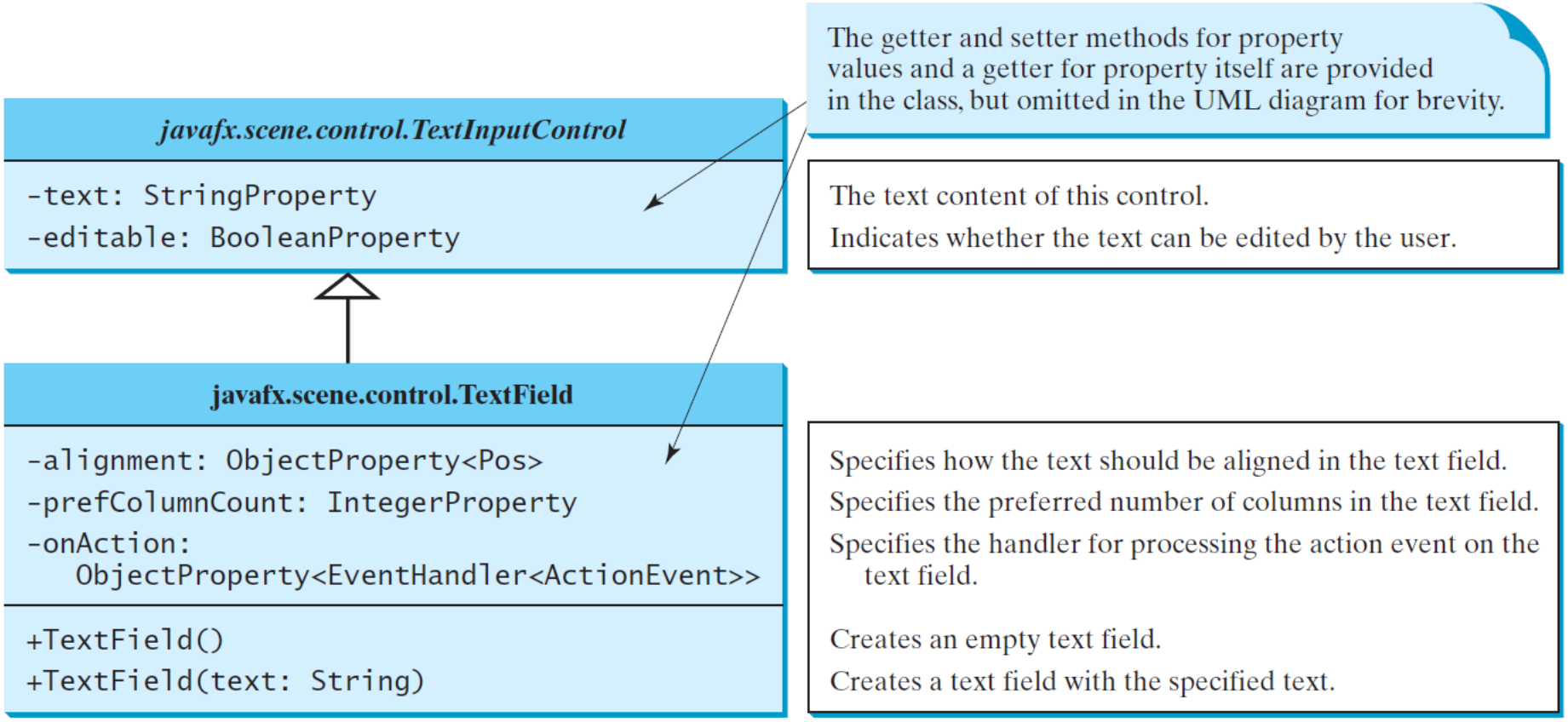

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```
    rbGreen.setOnAction(e -> {
      if (rbGreen.isSelected()) {
        text.setFill(Color.GREEN);
      }
    });

    rbBlue.setOnAction(e -> {
      if (rbBlue.isSelected()) {
        text.setFill(Color.BLUE);
      }
    });

    return pane;
  }
  // the start method is inherited from the superclass ButtonDemo
  public static void main(String[] args) {
    launch(args);
  }

}
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# TextField

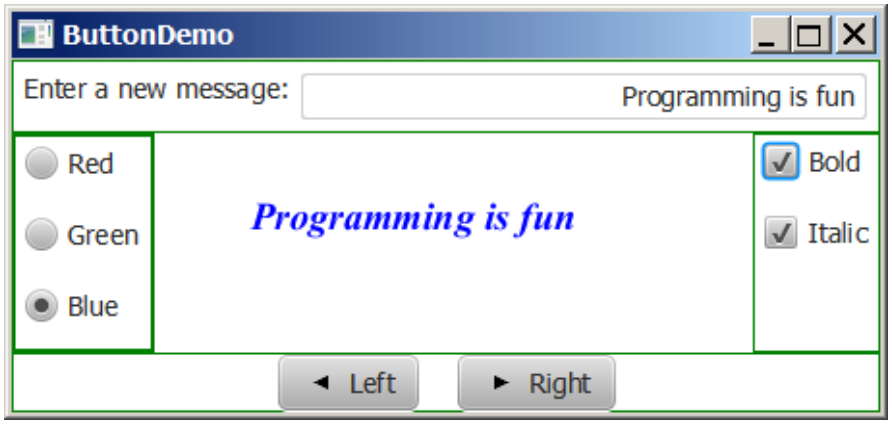
A text field can be used to enter or display a string. **TextField** is a subclass of **TextInputControl**.





Copyright © 2022 Pearson Education Ltd. All rights reserved.

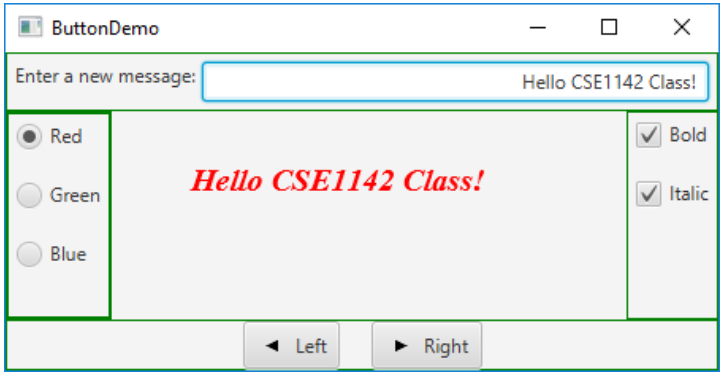
# TextField Example




TextFieldDemo


Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import static javafx.application.Application.launch;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.BorderPane;

public class TextFieldDemo extends RadioButtonDemo{
  @Override
  protected BorderPane getPane() {
    BorderPane pane = super.getPane();
    BorderPane paneForTextField = new BorderPane();
    paneForTextField.setPadding(new Insets(5, 5, 5, 5));
    paneForTextField.setStyle("-fx-border-color: green");
    paneForTextField.setLeft(new Label("Enter a new message: "));
    TextField tf = new TextField();
    tf.setAlignment(Pos.BOTTOM_RIGHT);
    paneForTextField.setCenter(tf);
    pane.setTop(paneForTextField);
    tf.setOnAction(e -> text.setText(tf.getText()));

    return pane;
  }
  public static void main(String[] args) {
    launch(args);
  }

}
```





Copyright © 2022 Pearson Education Ltd. All rights reserved.

# TextArea

A **TextArea** enables the user to enter multiple lines of text.

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

***javafx.scene.control.TextInputControl***

```
-text: StringProperty
-editable: BooleanProperty
```

The text content of this control.

Indicates whether the text can be edited by the user.

**javafx.scene.control.TextArea**

```
-prefColumnCount: IntegerProperty
-prefRowCount: IntegerProperty
-wrapText: BooleanProperty
```

Specifies the preferred number of text columns.

Specifies the preferred number of text rows.

Specifies whether the text is wrapped to the next line.

```
+TextArea()
+TextArea(text: String)
```

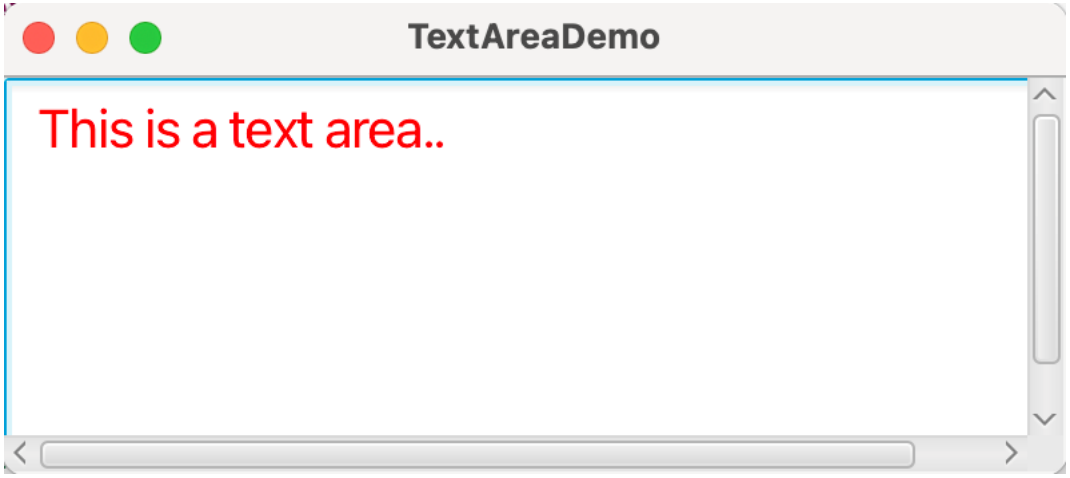
Creates an empty text area.

Creates a text area with the specified text.


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# TextArea Example

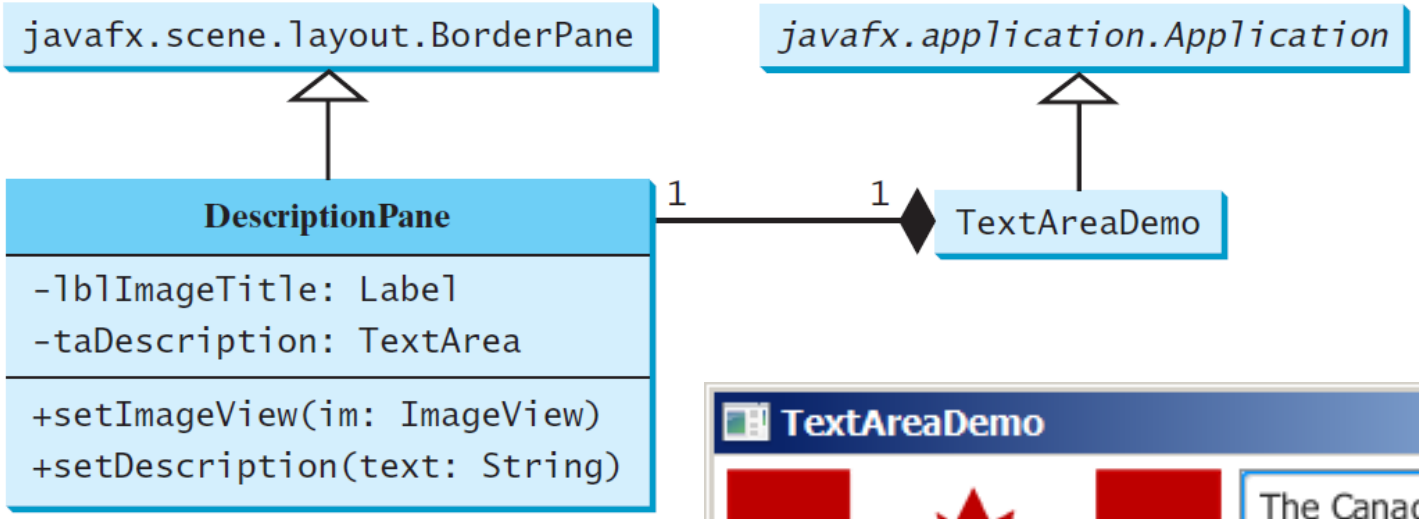
```
TextArea taNote = new TextArea("This is a text area..");
taNote.setPrefColumnCount(20);
taNote.setPrefRowCount(5);
taNote.setWrapText(true);
taNote.setStyle("-fx-text-fill: red");
taNote.setFont(Font.font("Times", 20));
ScrollPane pane = new ScrollPane(taNote);
```





Copyright © 2022 Pearson Education Ltd. All rights reserved.

# TextArea Example



DescriptionPane TextAreaDemo


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# ComboBox

A combo box, also known as a choice list or drop-down list, contains a list of items from which the user can choose.

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

***javafx.scene.control.ComboBoxBase<T>***

```
-value: ObjectProperty<T>
-editable: BooleanProperty
-onAction:
   ObjectProperty<EventHandler<ActionEvent>>
```

The value selected in the combo box.

Specifies whether the combo box allows user input.

Specifies the handler for processing the action event.

**javafx.scene.control.ComboBox<T>**

```
-items: ObjectProperty<ObservableList<T>>
-visibleRowCount: IntegerProperty
```

```
+ComboBox()
+ComboBox(items: ObservableList<T>)
```

The items in the combo box popup.

The maximum number of visible rows of the items in the combo box popup.

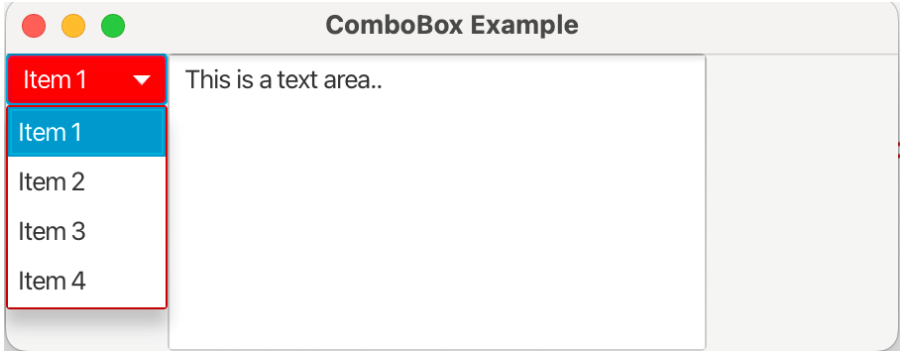
Creates an empty combo box.

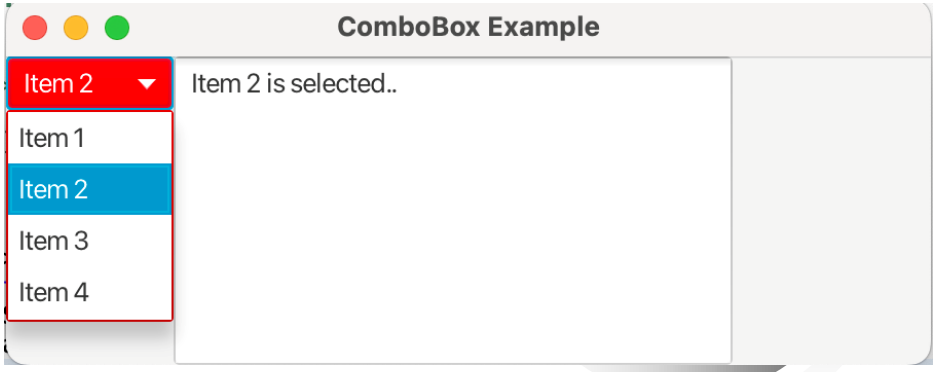
Creates a combo box with the specified items.


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# ComboBox Example

```
String options[] = { "Item 1", "Item 2", "Item 3", "Item 4" };
String messages[] = { "Item 1 is selected..", "Item 2 is selected..", "Item 3 is
selected..", "Item 4 is selected.." };

ComboBox<String> cbo = new ComboBox<>();
cbo.setStyle("-fx-color: red");
ObservableList<String> items = FXCollections.observableArrayList(options);
cbo.getItems().addAll(items); // Add items to combo box
cbo.setValue(items.get(0));

TextArea taNote = new TextArea("This is a text area..");
taNote.setPrefColumnCount(20);
taNote.setPrefRowCount(5);

cbo.setOnAction(e -> taNote.setText(messages[items.indexOf(cbo.getValue())]));
```





Copyright © 2022 Pearson Education Ltd. All rights reserved.

# ComboBox Example

This example lets users view an image and a description of a country's flag by selecting the country from a combo box.

ComboBoxDemo


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# ListView

A *list view* is a component that performs basically the same function as a combo box, but it enables the user to choose a single value or multiple values.

**javafx.scene.control.ListView<T>**

```
-items: ObjectProperty<ObservableList<T>>
-orientation: BooleanProperty

-selectionModel:
   ObjectProperty<MultipleSelectionModel<T>>

+ListView()
+ListView(items: ObservableList<T>)
```

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The items in the list view.

Indicates whether the items are displayed horizontally or vertically in the list view.

Specifies how items are selected. The `SelectionModel` is also used to obtain the selected items.

Creates an empty list view.

Creates a list view with the specified items.


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Example: Using ListView

This example gives a program that lets users select countries in a list and display the flags of the selected countries in the labels.

ListViewDemo


Copyright © 2022 Pearson Education Ltd. All rights reserved.

```
// import statements…

public class ListViewDemo extends Application {
  // Declare an array of Strings for flag titles
  private String[] flagTitles = {"United States of America", "Canada", "China",
"Denmark", "France", "Germany", "India"};
  // Declare an ImageView array for the national flags
  private ImageView[] ImageViews = {
    new ImageView("image/us.gif"),
    new ImageView("image/ca.gif"),
    new ImageView("image/china.gif"),
    new ImageView("image/denmark.gif"),
    new ImageView("image/fr.gif"),
    new ImageView("image/germany.gif"),
    new ImageView("image/india.gif")
  };
  @Override
  public void start(Stage primaryStage) {
    ListView<String> lv = new ListView<>(FXCollections
                                          .observableArrayList(flagTitles));

    lv.setPrefSize(400, 400);

    lv.getSelectionModel().setSelectionMode(SelectionMode.MULTIPLE);
```

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```
    // Create a pane to hold image views
    FlowPane imagePane = new FlowPane(10, 10);
    BorderPane pane = new BorderPane();
    pane.setLeft(new ScrollPane(lv));
    pane.setCenter(imagePane);

    lv.getSelectionModel().selectedItemProperty().addListener(
      ov -> {
        imagePane.getChildren().clear();
        for (Integer i: lv.getSelectionModel().getSelectedIndices()) {
          imagePane.getChildren().add(ImageViews[i]);
        }
    });

    Scene scene = new Scene(pane, 450, 170);
    primaryStage.setTitle("ListViewDemo");
    primaryStage.setScene(scene);
    primaryStage.show();
  }
  public static void main(String[] args) {
    launch(args);
  }

}
```

Copyright © 2022 Pearson Education Ltd. All rights reserved.

# ScrollBar

A *scroll bar* is a control that enables the user to select from a range of values. The scrollbar appears in two styles: *horizontal* and *vertical*.

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

**javafx.scene.control.ScrollBar**

```
-blockIncrement: DoubleProperty
-max: DoubleProperty
-min: DoubleProperty
-unitIncrement: DoubleProperty

-value: DoubleProperty
-visibleAmount: DoubleProperty
-orientation: ObjectProperty<Orientation>

+ScrollBar()
+increment()
+decrement()
```

The amount to adjust the scroll bar if the track of the bar is clicked (default: 10).

The maximum value represented by this scroll bar (default: 100).

The minimum value represented by this scroll bar (default: 0).

The amount to adjust the scroll bar when the `increment()` and `decrement()` methods are called (default: 1).

Current value of the scroll bar (default: 0).

The width of the scroll bar (default: 15).

Specifies the orientation of the scroll bar (default: `HORIZONTAL`).

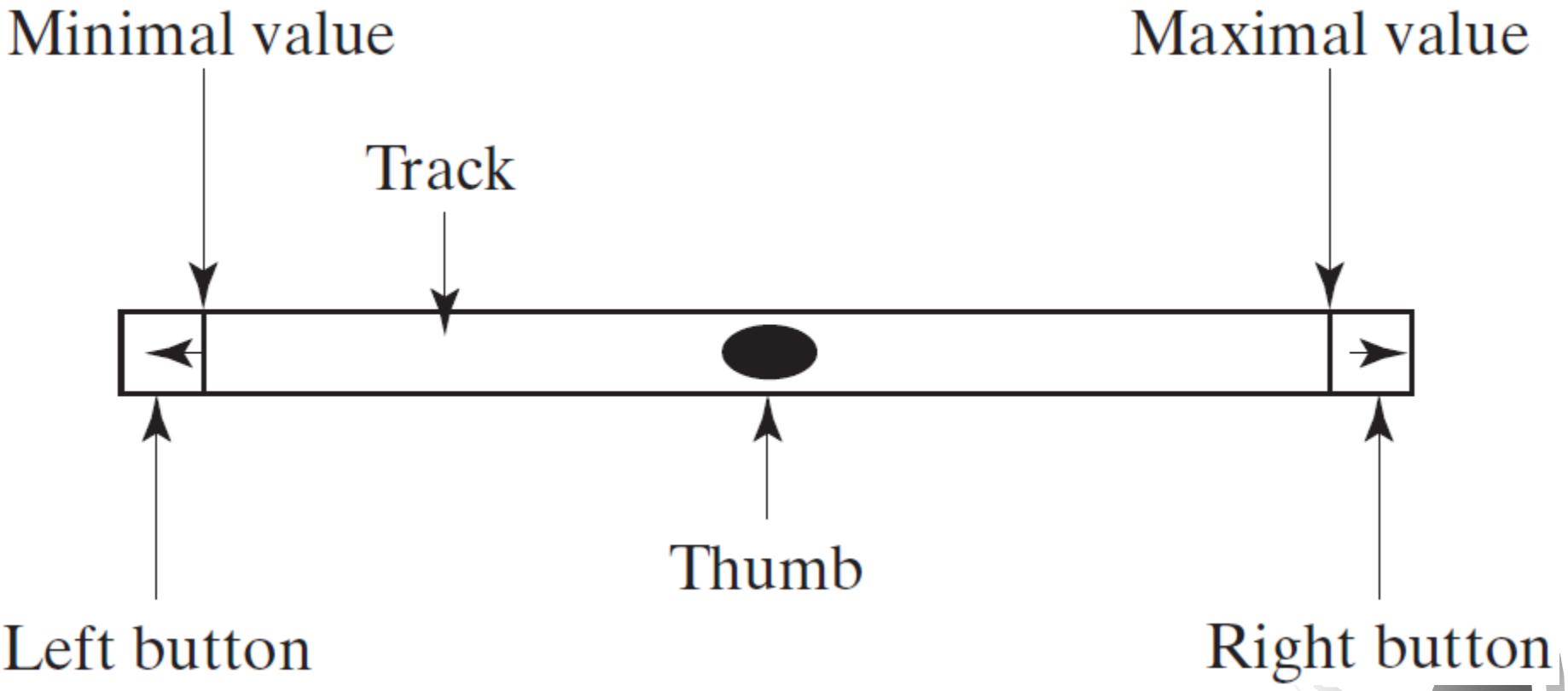
Creates a default horizontal scroll bar.

Increments the value of the scroll bar by `unitIncrement`.

Decrements the value of the scroll bar by `unitIncrement`.


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Scroll Bar Properties

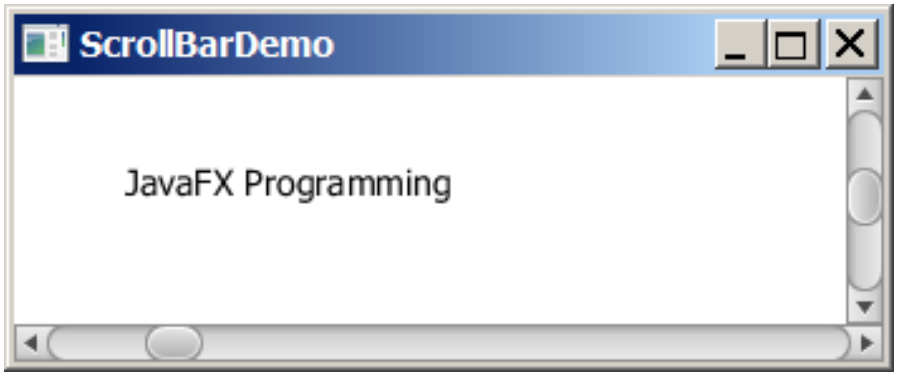




Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Example: Using Scrollbars

This example uses horizontal and vertical scrollbars to control a message displayed on a panel. The horizontal scrollbar is used to move the message to the left or the right, and the vertical scrollbar to move it up and down.




ScrollBarDemo


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Slider

Slider is similar to ScrollBar, but Slider has more properties and can appear in many forms.

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

**javafx.scene.control.Slider**

```
-blockIncrement: DoubleProperty
-max: DoubleProperty
-min: DoubleProperty
-value: DoubleProperty
-orientation: ObjectProperty<Orientation>
-majorTickUnit: DoubleProperty
-minorTickCount: IntegerProperty
-showTickLabels: BooleanProperty
-showTickMarks: BooleanProperty
```

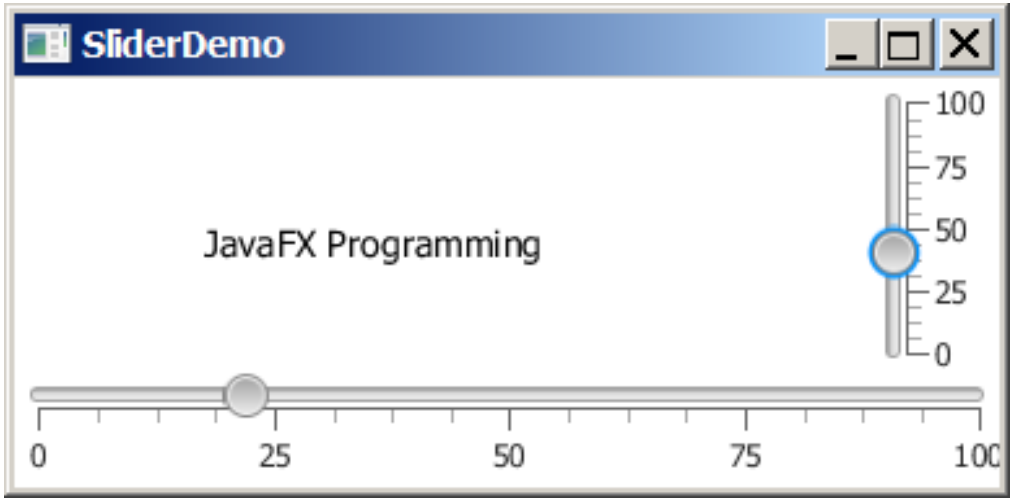
```
+Slider()
+Slider(min: double, max: double,
   value: double)
```

| | |
|---|---|
| -blockIncrement | The amount to adjust the slider if the track of the bar is clicked (default: 10). |
| -max | The maximum value represented by this slider (default: 100). |
| -min | The minimum value represented by this slider (default: 0). |
| -value | Current value of the slider (default: 0). |
| -orientation | Specifies the orientation of the slider (default: HORIZONTAL). |
| -majorTickUnit | The unit distance between major tick marks. |
| -minorTickCount | The number of minor ticks to place between two major ticks. |
| -showTickLabels | Specifies whether the labels for tick marks are shown. |
| -showTickMarks | Specifies whether the tick marks are shown. |
| +Slider() | Creates a default horizontal slider. |
| +Slider(min: double, max: double, value: double) | Creates a slider with the specified min, max, and value. |


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Example: Using Sliders

Rewrite the preceding program using the sliders to control a message displayed on a panel instead of using scroll bars.

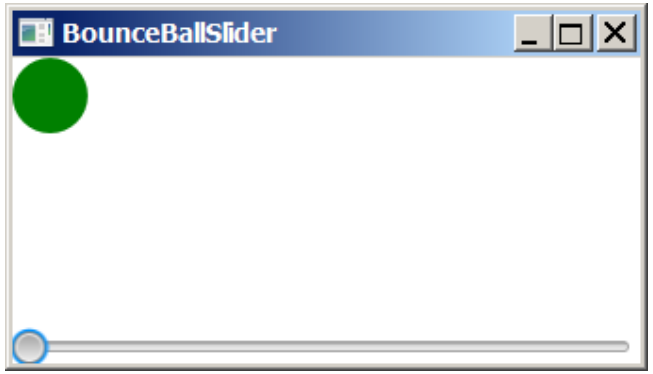


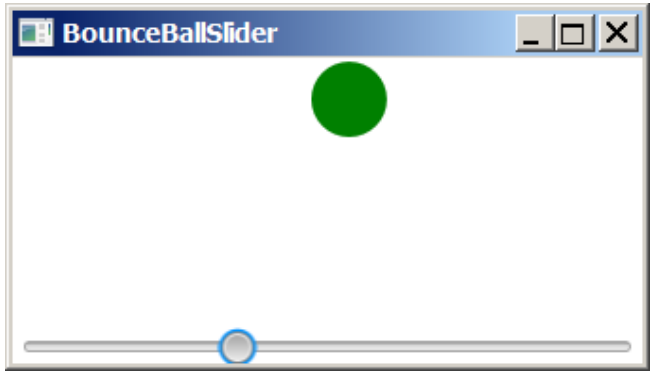


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Case Study: Bounce Ball

Listing 15.17 gives a program that displays a bouncing ball. You can add a slider to control the speed of the ball movement.

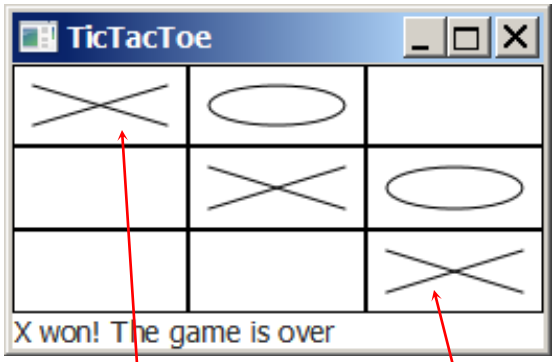


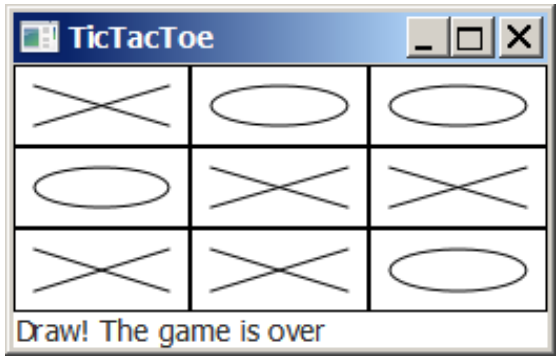



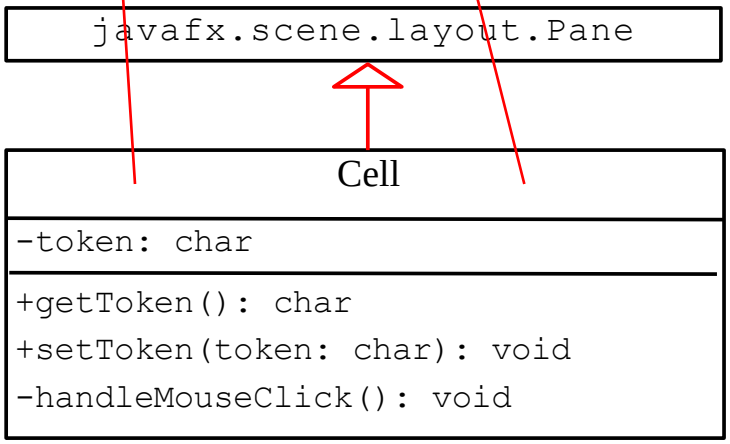

SliderDemo


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Case Study: TicTacToe




Token used in the cell (default: ' ').

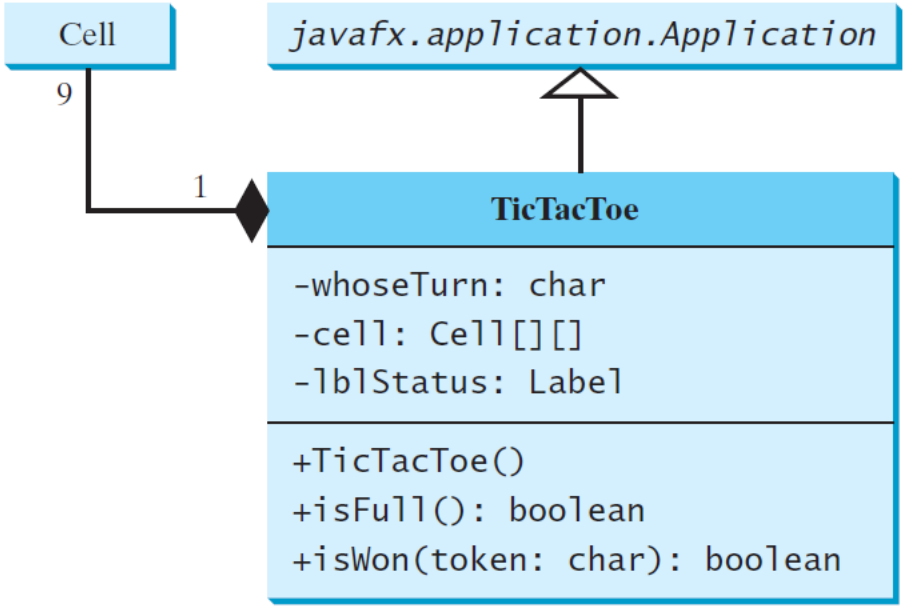
Returns the token in the cell.

Sets a new token in the cell.

Handles a mouse click event.


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Case Study: TicTacToe, cont.




Indicates which player has the turn, initially X.
A 3 × 3, two-dimensional array for cells.
A label to display game status.

Constructs the TicTacToe user interface.
Returns true if all cells are filled.
Returns true if a player with the specified token has won.

TicTacToe


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Media

You can use the **Media** class to obtain the source of the media, the **MediaPlayer** class to play and control the media, and the **MediaView** class to display the video.

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

**javafx.scene.media.Media**

```
-duration: ReadOnlyObjectProperty
   <Duration>
-width: ReadOnlyIntegerProperty
-height: ReadOnlyIntegerProperty
+Media(source: String)
```

The durations in seconds of the source media.

The width in pixels of the source video.

The height in pixels of the source video.

Creates a `Media` from a URL source.


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# MediaPlayer

The **MediaPlayer** class playes and controls the media with properties such as **autoPlay**, **currentCount**, **cycleCount**, **mute**, **volume**, and **totalDuration**.

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

| javafx.scene.media.MediaPlayer | |
|---|---|
| -autoPlay: BooleanProperty | Specifies whether the playing should start automatically. |
| -currentCount: ReadOnlyIntegerProperty | The number of completed playback cycles. |
| -cycleCount: IntegerProperty | Specifies the number of time the media will be played. |
| -mute: BooleanProperty | Specifies whether the audio is muted. |
| -volume: DoubleProperty | The volume for the audio. |
| -totalDuration: ReadOnlyObjectProperty<Duration> | The amount of time to play the media from start to finish. |
| +MediaPlayer(media: Media) | Creates a player for a specified media. |
| +play(): void | Plays the media. |
| +pause(): void | Pauses the media. |
| +seek(): void | Seeks the player to a new playback time. |


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# MediaView

The **MediaView** class is a subclass of **Node** that provides a view of the **Media** being played by a **MediaPlayer**. The **MediaView** class provides the properties for viewing the media.

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

**javafx.scene.media.MediaView**

```
-x: DoubleProperty
-y: DoubleProperty
-mediaPlayer:
   ObjectProperty<MediaPlayer>
-fitWidth: DoubleProperty
-fitHeight: DoubleProperty
```

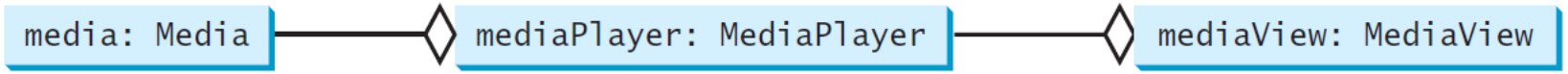
```
+MediaView()
+MediaView(mediaPlayer: MediaPlayer)
```

| | |
|---|---|
| -x: DoubleProperty | Specifies the current x-coordinate of the media view. |
| -y: DoubleProperty | Specifies the current y-coordinate of the media view. |
| -mediaPlayer: ObjectProperty<MediaPlayer> | Specifies a media player for the media view. |
| -fitWidth: DoubleProperty | Specifies the width of the view for the media to fit. |
| -fitHeight: DoubleProperty | Specifies the height of the view for the media to fit. |
| +MediaView() | Creates an empty media view. |
| +MediaView(mediaPlayer: MediaPlayer) | Creates a media view with the specified media player. |


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Example: Using Media

This example displays a video in a view. You can use the play/pause button to play or pause the video and use the rewind button to restart the video, and use the slider to control the volume of the audio.

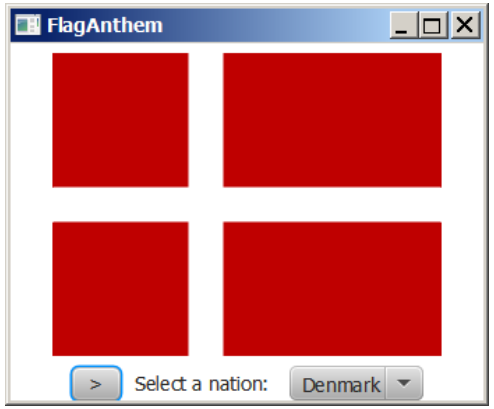



MediaDemo


Copyright © 2022 Pearson Education Ltd. All rights reserved.

# Case Study: National Flags and Anthems

This case study presents a program that displays a nation's flag and plays its anthem.



FlagAnthem


Copyright © 2022 Pearson Education Ltd. All rights reserved.